

CHAPTER 5

Verilog 的敘述

Verilog

本章主要是介紹：Verilog 常用的敘述，包括：parameter、assign、always、if、if ... else ...、case、casex、casez、for、function、task...等敘述。我們也簡單扼要的說明 if 與 case 這二大類敘述的使用時機、還有 function 與 task 敘述的差異比較...等等。



5.1 Verilog 常用的敘述

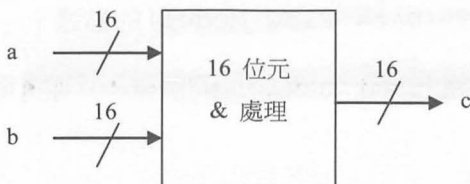
在模組中，使用關鍵字 parameter 來定義一個常數，對於可擴性設計的模組是非常好用的。

BNF 表示法

```
<parameter_declaration>
:= parameter <range>? <list_of_assignments>;
```

範例

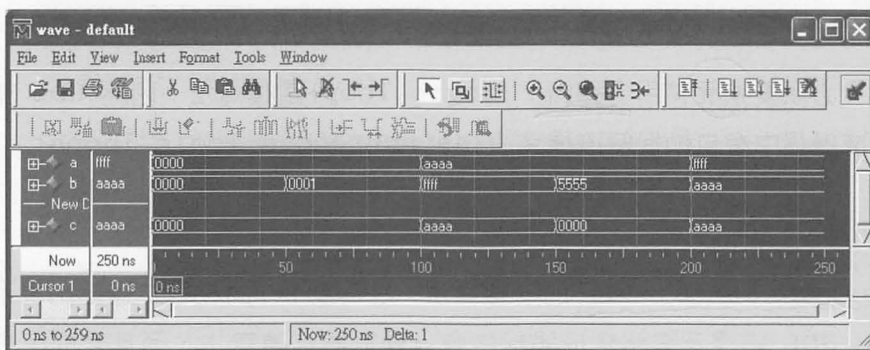
以底下的這個例子來說，ScalableDesign 模組的功能為處理 2 個輸入資料 a 與 b 作 16 個位元的 & 動作後，將結果給 c。在第 3 行中有 parameter 這個敘述，且設定了一個名為 width 的常數，值為 16；在第 4 行及第 5 行中分別引用了 input [width-1:0] a, b; output [width-1:0] c; 相當於 input [15:0] a, b; output [15:0] c; 我們只要把在第 3 行中 parameter 這個敘述所指定之常數 width 的值改成所需要的為位元組長度即可達到易於修改的目的，例如：將 width = 16 改成 width = 32，ScalableDesign 模組的功能就改成了為處理 2 個輸入資料 a 與 b 作 32 個位元的 & 動作後，將結果給 c。



▲ EasyScal.V：易於修改的模組設計，使用 parameter 敘述的邏輯符號

```
// EasyScal.V, 易於修改的模組設計, 使用 parameter 敘述
module ScalableDesign(a, b, c);
parameter width = 16;
input [width-1:0] a, b;
output [width-1:0] c;
wire [width-1:0] c;

assign c = a & b;
endmodule
```



▲ EasyScal.V 的模擬結果



5.2 assign 敘述

assign 敘述的功能，如下：

- 驅動某個值到 wire、wand、wor 或 tri。

- 用於資料處理模型 (Data Flow Model) 的描述。
- 用於描述組合邏輯 (Combinational circuits) 電路。

BNF 表示法

```
<continuous_assignment>
:= assign <drive_strength>? <list_of_assignments>;
```

例 1

```
wire a, b, c; // 宣告三個接線型態的變數，分別為：a, b 及 c
assign a = b & c; // 指定 a = b 和 c 作 & 運算

wire b, c; // 宣告二個接線型態的變數，分別為：a 及 b
wire a = b & c; // 宣告並且指定 a = b 和 c 作 & 運算
```

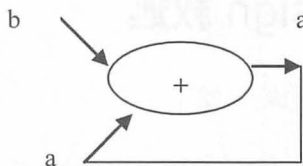
assign 敘述的問題，如下：

我們必須要避免使用「迴路」(logic loop)這種寫法，因為 HDL 編譯及電路合成過程中會自動拆解這種寫法 (asynchronous logic loops)。

例 2

assign a = b + a; 的這種寫法，在等號(=)的左右二邊都出現了相同的變數，HDL 編譯及電路合成過程中會自動拆解這種寫法，而且合成出來的電路在電路的模擬擬時會發生時序上的錯誤、也就是電路的模擬結果會有不預期的結果出現。

```
assign a = b + a;
```





5.3 always 敘述

`always` 敘述的觀念有點像是一名警衛一樣，隨時在監督外界（輸出入埠）訊號的情況，當外界的訊號一有變化隨即告知模組內部要配合作相對應的處理。

BNF 表示法

```
<always_instantiation>
:= always <statement> ||
   always <seq_block>
```

或

```
always @(event-expression [or event-expression*]
begin
  <statement_or_null>
end
```

Synopsys 會使用什麼元件來合成出 Gate Level 電路呢？

（一）情形一

在 `event-expression` (事件表示式) 中的變數必須在 `always @` 的 `begin ... end` 這個 block 中出現，否則的話 Synopsys 將會使用 **Latches** 來合成出 Gate Level 電路。

例 1 準位觸發 (Level Trigger)

```
reg a, b, c, f;
always @(a or b or c) // 只要訊號 a, b 或者是 c 有變化 (由 0→1 或 1→0)
begin
  f = a & b & c; // 則更新 f 的值, f = a & b & c
end
```



▲ 訊號由 0 轉變為 1 (0 → 1)



▲ 訊號由 1 轉變為 0 (1 → 0)

例 2 準位觸發 (Level Trigger)

```
reg [7:0] m, n, k;
always @(m or n) // 只要訊號 m 或者是 n 的值有任何的變化，就會重算 k
begin
    // (都是 8 個位元，0 ~ 255)
    k = m + n; // 則更新 k 的值，k = m + n
end
```

(一) 情形二

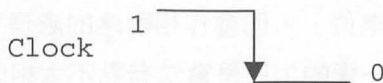
在 event-expression 中訊號名稱的前面，如果加上 **posedge** edge trigger (正緣觸發) 或者是 **negedge** edge trigger (負緣觸發) 這二個識別字 (Keyword) 的其中一種，則 Synopsys 將會使用 **flip-flop** 來合成出 Gate Level 電路。

例 3 posedge 正緣觸發

```
always @(posedge Clock) // 只要訊號 Clock 為正緣觸發
begin
    f = a & b & c; // 則更新 f 的值，f = a & b & c
end
```



▲ 訊號 Clock 為 posedge edge trigger (正緣觸發)

例 4 negedge 負緣觸發

▲ 訊號 Clock 爲 negedge edge trigger (負緣觸發)

```
always @(negedge Clock) // 只要訊號 Clock 爲負緣觸發
begin
    f = a & b & c; // 則更新 f 的值, f = a & b & c
end
```

除了上述的二種情形以外，Synopsys 將會使用組合電路 (Combinational Logic) 來合成出 Gate Level 電路。



5.4 if 敘述

if 敘述爲一種二路分支選擇的條件式判斷敘述，通常是用來作為判斷訊號的值之後、再選擇要作的處理。

BNF 表示法

```
if ( <expression> )
    <statement_or_null>
```

或

```
if (<expression> )
begin
    <statement_or_null>
end
```

if 敘述能處理「正準位」觸發 (Positive Level Trigger) 以及「負準位」觸發 (Negative Level Trigger) 等二種訊號。

(一) 正準位觸發 (Positive Level Trigger)

只要訊號的值是「正準位」，則會作相對應的處理。在例 1 和例 2 中：if 敘述內的判斷動作是完全一樣的，只是寫法有點不太相同而已。

例 1 正準位觸發 (Positive Level Trigger) 的第一種寫法

```
always @(posedge Clock)
begin
    if (Reset == 1'b1) // 如果 Reset 的值等於 1
        Out = 0; // 則令 Out = 0
end
```

例 2 正準位觸發 (Positive Level Trigger) 的第二種寫法

```
always @(posedge Clock)
begin
    if (Reset) // 如果 Reset 的值等於 1
        Out = 0; // 則令 Out = 0
end
```

(二) 負準位觸發 (Negative Level Trigger)

只要訊號的值是「負準位」，則會作相對應的處理。在例 3 和例 4 中：if 敘述內的判斷動作是完全一樣的，只是寫法有點不太相同而已。

例 3 負準位觸發 (Negative Level Trigger) 的第一種寫法

```
always @(posedge Clock)
begin
    if (Reset == 1'b0) // 如果 Reset 的值等於 0
        Out = 0; // 則令 Out = 0
    else
        Out = In_Data;
end
```


例 4 負準位觸發 (Negative Level Trigger) 的第二種寫法

```

always @(posedge Clock)
begin
  if (!Reset)    // 如果 Reset 的值等於 0
    Out = 0;     // 則令 Out = 0
  else
    Out = In_Data;
end

```

**5.5 if ... else ... 敘述**

if ... else ... 敘述為一種二路分支選擇的條件式判斷敘述，表面上的意思是：如果... / 否則 ... 敘述。

BNF 表示法

```

if ( <expression> )
  <statement_or_null>
else
  <statement_or_null>

```

或

```

if (<expression> )
begin
  <statement_or_null>
end
else
begin
  <statement_or_null>
end

```

實例：底下的範例中功能是相同的，只是描述的語法不同。Synopsys 將會使用 **Priority Encoder** 來合成出 Gate Level 電路。

例 1

```
always @(X or Y or Z)
begin
    Out = Result1;           // 優先權 (Priority) 最低
    if (X)                   // 優先權 (Priority) 第 3 高
        Out = Result2;
    if (Y)                   // 優先權 (Priority) 第 2 高
        Out = Result3;
    if (Z)                   // 優先權 (Priority) 最高
        Out = Result4;
end
```

例 2

```
always @(X or Y or Z)
begin
    if (Z)                   // 優先權 (Priority) 最高
        Out = Result4;
    else
        if (Y)               // 優先權 (Priority) 第 2 高
            Out = Result3;
        else
            if (X)            // 優先權 (Priority) 第 3 高
                Out = Result2;
            else
                Out = Result1; // 優先權 (Priority) 最低
end
```



5.6 case 敘述

case 敘述為一種多路分支選擇的敘述。如果電路中所有可能的分支判別條件都被指定了，則稱為 **full case**。

BNF 表示法

```
<case_item>
```

```

:= <expression> <, <expression>>* : <statement_or_null> ||
default : <statement_or_null> ||
default <statement>

```

或

```

case (expr)
  case_item1 : begin
    <statement_or_null>
  end
  case_item2 : begin
    <statement_or_null>
  end
  default : begin
    <statement_or_null>
  end
endcase

```

例 1

```

always @(Select or in1 or in2 or in3 or in4)
begin
  case (Select)
    2'b00: Out = in1;
    2'b01: Out = in2;
    2'b10: Out = in3;
    2'b11: Out = in4;
    default: Out = 1'bx;
  endcase
end

```

如果沒有將“所有可能的”分支判別條件都指定，則 Synopsys 將會使用 **Latch** 來合成出 Gate Level 電路。

例 2 少了 Select = 2'b10 及 Select = default 的指定。

```

always @(Select or in1 or in2 or in4)
begin
  case (Select)
    2'b00: Out = in1;
    2'b01: Out = in2;

```

```

        2'b11: Out = in4;
    endcase
end

```

如果您沒有將“所有可能的”分支判別條件都指定，但您知道其他您所未指定的條件結果都不可能發生，則您可以使用 `// synopsys full_case` 編譯命令 (Compiler Directives) 將您的電路指定為 **full case** 的特性，也就是自動將沒用的情況為忽略 (don't care)。用以防止 Synopsys 使用 Latch 而使用 **Priority Encoder** 來合成出 Gate Level 電路。

例 3

```

always @(Select or in1 or in2 or in4)
begin // synopsys full_case
    case (Select)
        2'b00: Out = in1;
        2'b01: Out = in2;
        2'b11: Out = in4;
    endcase
end

```

注意 NOTICE

Xilinx Vivado 是看不懂 `// synopsys full_case` 這個編譯命令的，也就是說上述的範例只適用在 Synopsys 的 HDL 編譯器。

如果 HDL Compiler 無法決定這個 case 是 parallel，則 Synopsys 將會使用 **Priority Decoder** 來合成出 Gate Level 電路。

底下的範例中功能是相同的，只是描述的語法不同，且 Synopsys 將會使用 **Priority Decoder** 來合成出 Gate Level 電路。

例 4

```

always @(u or v or w)
begin
    case (2'b11)

```

```
u: Out = 3'b001; // 優先權 (Priority) 最高
v: Out = 3'b010; // 優先權 (Priority) 第 2 高
w: Out = 3'b100; // 優先權 (Priority) 最低
default: Out = 3'b000;
endcase
end 例 1:
```

例 5

```
always @(u or v or w)
begin
  if u = 2'b11 // 優先權 (Priority) 最高
    Out = 3'b001;
  else
    if v = 2'b11 // 優先權 (Priority) 第 2 高
      Out = 3'b010;
    else
      if w = 2'b11 // 優先權 (Priority) 最低
        Out = 3'b100;
      else
        Out = 3'b000;
end
```

可以使用 `// synopsys parallel_case` 將您的電路指定為 parallel case 的特性，這樣 Synopsys 就不會使用 Priority Decoder 來合成出 Gate Level 電路，而會產生 Parallel Decoder 電路。

例 6

```
always @(select)
begin // synopsys parallel_case
  case (Select)
    2'b00: Out = in1;
    2'b01: Out = in2;
    2'b11: Out = in4;
  endcase
end
```

例 7

```
always @(X or Y or Z)
begin
    temp = {X, Y, Z};
    casex (temp) // synopsys parallel_case
        3'b1xx: Value = Result1; // X = 1
        3'bx1x: Value = Result2; // Y = 1
        3'bxx1: Value = Result3; // Z = 1
        default: Value = Result4; // others
    endcase
end
```

例 8

```
always @(X or Y or Z)
begin
    case (1'b1) // synopsys parallel_case
        X: Value = Result1; // X = 1
        Y: Value = Result2; // Y = 1
        Z: Value = Result3; // Z = 1
        default: Value = Result4; // others
    endcase
end
```

注意 NOTICE

Xilinx Vivado 是看不懂 `// synopsys parallel_case` 這個編譯命令的，也就是說上述的範例只適用在 Synopsys 的 HDL 編譯器。



5.7 casex 敘述

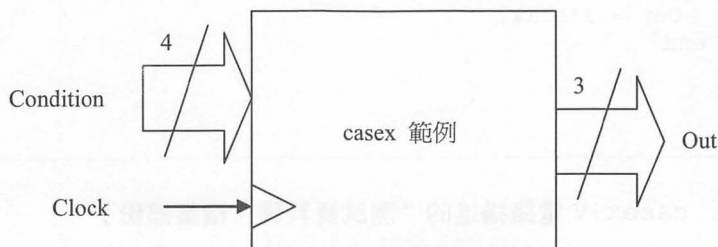
- 允許 `case_item` 的值為「？」，在“測試資料碼”時，「？」可以用 0 或 1 來當作測試的值。請參考 `casex.V` 電路描述及 `casex.tf` “測試資料碼”檔案。
- 不允許 `expr` 中有 `z` 或 `x`。

BNF 表示法

```

case (expr)
  case_item1 : begin
    <statement_or_null>
  end
  case_item2 : begin
    <statement_or_null>
  end
  default : begin
    <statement_or_null>
  end
endcase

```



▲ case 敘述，範例的邏輯符號

【casex.V: casex 敘述，x：隨意值(Don't care)的模組電路描述部份】

```

// casex.V: casex 敘述，範例
// 我們希望時脈訊號 (Clock) 有正緣觸發時，
// 才判斷 Condition 值，決定輸出值 Out
module casex_machine (Clock, Condition, Out);
input      Clock;
input  [3:0] Condition;
output  [2:0] Out;
reg  [2:0] Out;
always @(posedge Clock) // 當時脈訊號有正緣觸發時
begin
  casex (Condition)
    4'b1??? : // Out = 3'b000, 只要 Condition[3] = 1
      begin
        Out = 3'b000;
      end
    4'b?1?? : // Out = 3'b001, 只要 Condition[2] = 1
      begin

```



```

        Out = 3'b001;
    end
    4'b???1?: // Out = 3'b010, 只要 Condition[1] = 1
    begin
        Out = 3'b010;
    end
    4'b???1: // Out = 3'b011, 只要 Condition[0] = 1
    begin
        Out = 3'b011;
    end
    4'b0000: // Out = 3'b100, 如果 Condition = 4'b0000
    begin
        Out = 3'b100;
    end
    default: // Out = 3'b111, Condition 為其他值
    begin
        Out = 3'b111;
    end
endcase
end
endmodule

```

【casex.tf, casex.V 電路描述的“測試資料碼”檔案部份】

// casex.tf: 我們截取 casex.V 電路描述的“測試資料碼”檔案部份

```

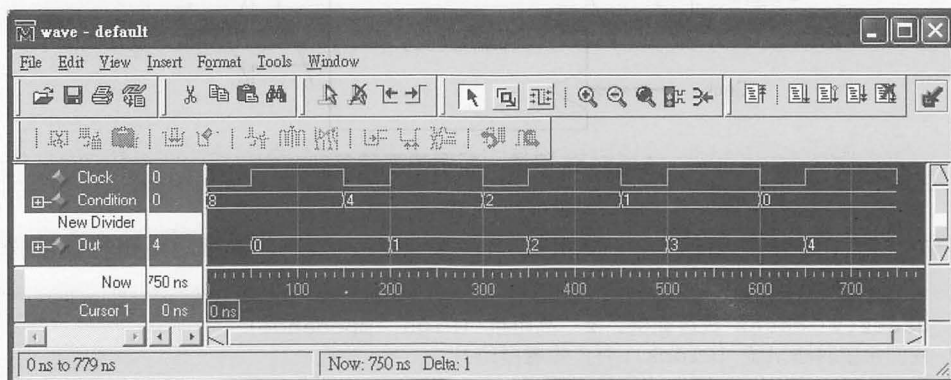
initial
begin
    Clock    = 1'b0;
    Condition = 4'b1000;
    #50
    Clock    = 1'b1;
    #100
    Clock    = 1'b0;
    Condition = 4'b0100;
    #50
    Clock    = 1'b1;
    #100
    Clock    = 1'b0;
    Condition = 4'b0010;
    #50
    Clock    = 1'b1;
    #100
    Clock    = 1'b0;
    Condition = 4'b0001;
    #50

```

```

        Clock    = 1'b1;
#100
        Clock    = 1'b0;
        Condition = 4'b0000;
#50
        Clock    = 1'b1;
end

```



▲ casex.V 的模擬結果



5.8 casez 敘述

允許 `case_item` 的值為「？」，在“測試資料碼”時，「？」可以用 0、1、? 或 z 來當作測試的值，也就是任意值均可。請參考 `casez.V` 電路描述及 `casez.tf` “測試資料碼”檔案。

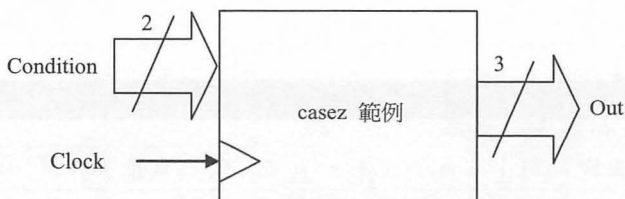
BNF 表示法

```

casez (expr)
  case_item1 : begin
    <statement_or_null>
  end
  case_item2 : begin
    <statement_or_null>
  end
end

```

```
default : begin
    <statement_or_null>
end
endcase
```



▲ casez 敘述，範例的邏輯符號

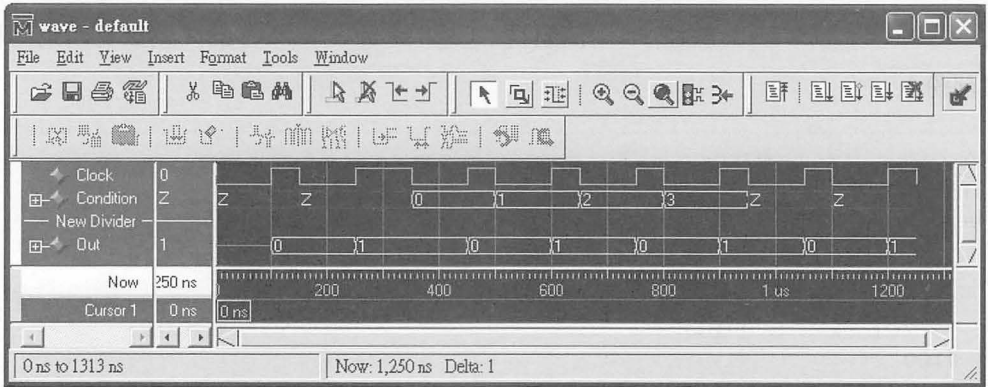
【casez.V: casez 敘述，z：高阻抗 (High Impedance) 的模組電路描述部份】

```
// casez.V: casez 敘述範例，z：高阻抗 (High Impedance)
// 我們希望時脈訊號 (Clock) 有正緣觸發時，
// 才判斷 Condition 值，決定輸出值 Out
module casez_machine (Clock, Condition, Out);
input    Clock;
input    [1:0] Condition;
output   [2:0] Out;
reg      [2:0] Out;

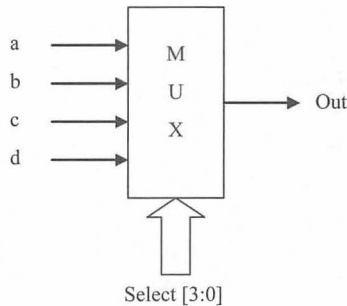
always @(posedge Clock) // 當時脈訊號有正緣觸發時
begin
    casez (Condition)
        2'bz0: // 只要 Condition[0] = 0, 則 Out = 3'b000;
            begin
                Out = 3'b000;
            end
        2'bz1: // 只要 Condition[0] = 1, 則 Out = 3'b001;
            begin
                Out = 3'b001;
            end
        default: // Condition 為其他值, 則 Out = 3'b111;
            begin
                Out = 3'b111;
            end
    endcase
end
endmodule
```

【casez.tf: casez 敘述, z: 高阻抗 (High Impedance) 的“測試資料碼”檔案部份】

```
// casez.tf: 我們截取 casez.V 電路描述的“測試資料碼”檔案部份
initial
begin
    Clock    = 1'b0;
    Condition = 2'bz0;
#100
    Clock    = 1'b1;
#100
    Clock    = 1'b0;
    Condition = 2'bz1;
#100
    Clock    = 1'b1;
#100
    Clock    = 1'b0;
    Condition = 2'b00;
#100
    Clock    = 1'b1;
#100
    Clock    = 1'b0;
    Condition = 2'b01;
#100
    Clock    = 1'b1;
#100
    Clock    = 1'b0;
    Condition = 2'b10;
#100
    Clock    = 1'b1;
#100
    Clock    = 1'b0;
    Condition = 2'b11;
#100
    Clock    = 1'b1;
#100
    Clock    = 1'b0;
    Condition = 2'b?0;
#100
    Clock    = 1'b1;
#50
    Clock    = 1'b0;
    Condition = 2'b?1;
#100
    Clock    = 1'b1;
end
```

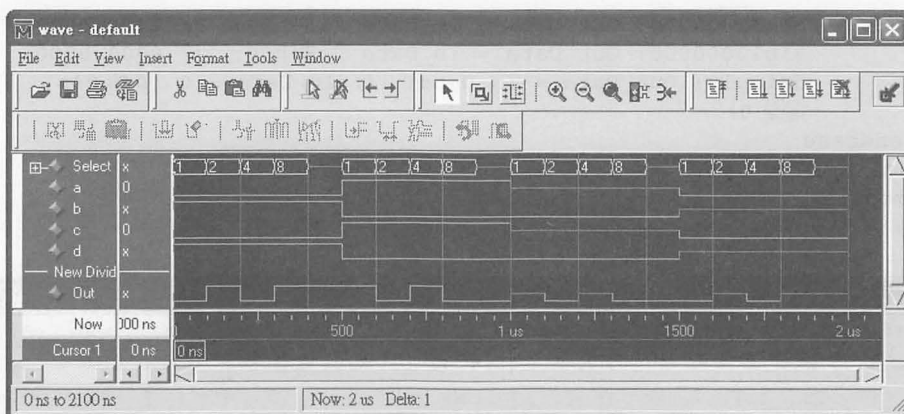


▲ casez.V 的模擬結果

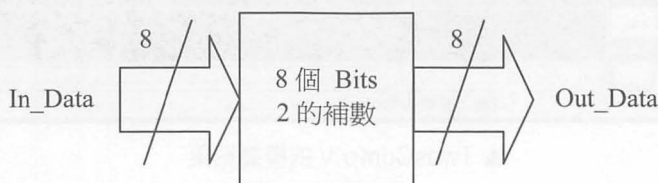


▲ M_casez.V : 4 對 1 的多工器

```
// M_casez.V : 4 對 1 的多工器
module Mux_casez (Select, a, b, c, d, Out);
input    [3:0] Select;
input    a, b, c, d;
output    Out;
reg       Out;
always @(Select or a or b or c or d)
begin
    casez (Select)
        4'b???1: Out = a;
        4'b??1?: Out = b;
        4'b?1??: Out = c;
        4'b1???: Out = d;
    endcase
end
endmodule
```



▲ Mux_casex.V 的模擬結果



▲ TwosComp.V : 8 個 Bits 的 2 的補數的邏輯符號

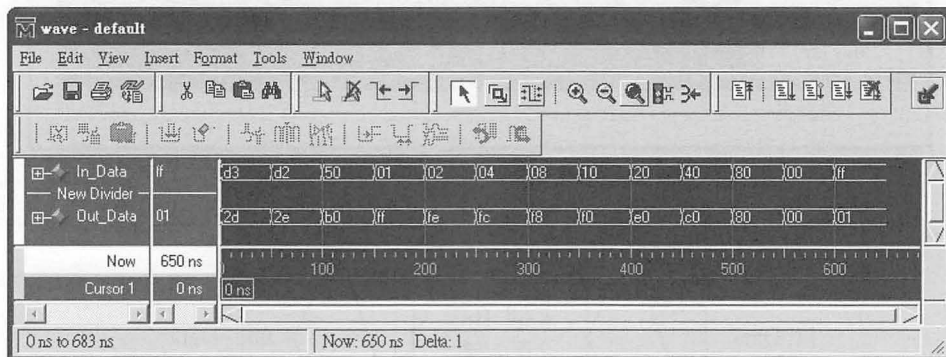
```
// TwosComp.V : 8 個 Bits 的 2 的補數
// 例 : In_Data = 1101_0011,    1101_0010,    0101_0000
//       XOR ^1111_1110,    ^1111_1100,    ^1110_0000
// 運算結果 = 0010_1101, = 0010_1110, =1011_0000
// → 等於 2's = 0010_1101, = 0010_1110, =1011_0000
module TwosComp (In_Data, Out_Data);
input    [7:0] In_Data;
output   [7:0] Out_Data;
reg      [7:0] Out_Data;
always @(In_Data)
begin
    casez (In_Data) //synopsys parallel_case
        8'b???????1: Out_Data = In_Data ^ 8'hfe; // 1111_1110
        8'b???????10: Out_Data = In_Data ^ 8'hfc; // 1111_1100
        8'b???????100: Out_Data = In_Data ^ 8'hf8; // 1111_1000
        8'b?????1000: Out_Data = In_Data ^ 8'hf0; // 1111_0000
        8'b???10000: Out_Data = In_Data ^ 8'he0; // 1110_0000
        8'b??100000: Out_Data = In_Data ^ 8'hc0; // 1100_0000
    endcase
end
```

```

8'b?1000000: Out_Data = In_Data ^ 8'h80; // 1000_0000
8'b10000000: Out_Data = In_Data ^ 8'h00; // 0000_0000
default: Out_Data = In_Data; // In_Data = 0 (0的2's,
還是 0)

endcase
end
endmodule

```



▲ TwosComp.V 的模擬結果



5.9 if 與 case 這二大類敘述的使用時機

一般對於 if 與 case 這二大類敘述的使用時機，通常都會作下述的考慮：

- 如果“if ... else ...”形成的描述太長了，則改用“case”敘述會比較清楚易懂。但並不是每個用“if ... else ...”形成的描述都能改用“case”敘述來描述的。
- 如果您知道您在“case”敘述中所描述的指定動作具有互斥 (mutual exclusive) 的特性時，請使用 `// synopsys parallel_case`，用以讓 Synopsys 使用 **Parallel Decoder** 來合成出 Gate Level 電路。互斥 (mutual exclusive) 的特性是指所有可能的分支判別情形均已放入各個 case 敘述的選項之中。

- 如果您沒有將所有可能的分支判別條件都指定，但您知道其他您所未指定的條件結果都不可能會發生，則您能使用 `// synopsys full_case` 將您的電路指定為 `full case` 的特性，也就是自動將沒用的情況為忽略 (`don't care`)。用以防止 Synopsys 使用 `Latch` 來合成出 Gate Level 電路。



5.10 for 敘述

`for` 敘述主要是提供一個簡單的方法來描述具有重覆特性的敘述。

BNF 表示法

```
for (<assignment> ; <expression> ; <assignment> )
begin
  <statement>
end
```

例如：

```
for (index = low_range; index < high_range; index = index + step)
```

或

```
for (index = high_range; index > low_range; index = index - step)
```

或

```
for (index = low_range; index <= high_range; index = index + step)
```

或

```
for (index = high_range; index >= low_range; index = index - step)
```


範例

```
always @(a or b)
begin: for_loop_test
  integer i;
  for (i = 0; i <= 3; i = i+1)
  begin
    Out[i] = a[i] & b[i];
    c = (a[i] | b[i]) & c;
  end
end
```

其中 for loops 內的敘述在編譯 (Compile Time) 時會被展開成等效的敘述，如下：

```
always @(a or b)
begin
  Out[0] = a[0] ^ b[0];
  Out[1] = a[1] ^ b[1];
  Out[2] = a[2] ^ b[2];
  Out[3] = a[3] ^ b[3];
  c = (a[0] | b[0]) & (a[1] | b[1]) & (a[2] | b[2]) & (a[3] | b[3])
    & c;
end
```

 5.11 function 敘述

在行為模型 (Behavioral Model) 的設計中、我們通常在程式的許多地方重複著相同描述的程式碼，這樣的情況在一般的電腦程式語言裡，通常將重複用到的程式碼設計成函數或是副程式的形式，再去引用，而不用繁瑣的鍵入同樣的程式碼，在 Verilog 中對於相同的程式碼可以在程式中的不同地方被引用而不需要繁瑣的一再鍵入相同的程式碼。

■ function 敘述是以關鍵字 function 與 endfunction 宣告。

- function 敘述如同在 Quick Basic 程式語言、Pascal 程式語言或 Fortran 程式語言裡的 function。
- function 敘述同樣是被包含在 Verilog 階層式的架構中，同樣可以用階層式的名稱來引用到它。
- function 的傳回值是儲存在和函數同樣名稱的這個變數中。

BNF 表示法

```

<function>
:= function <range>? <function_name> ;
  <function_declaration>*
  <statement_or_null>
endfunction

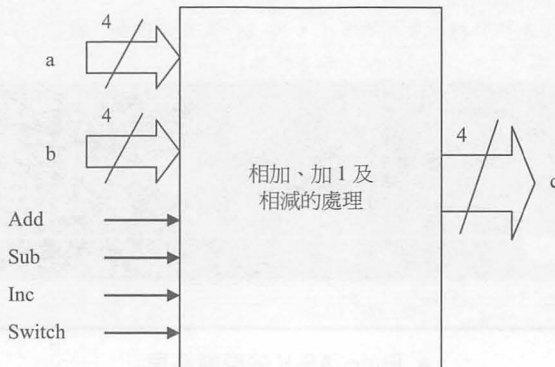
<function_declaration>
:= <parameter_declaration> ||
  <input_declaration> ||
  <reg_declaration> ||
  <integer_declaration>

<function_name>
:= <IDENTIFIER>

<function_call>
:= <function_name> ( <expression> <, <expression>>* )

```

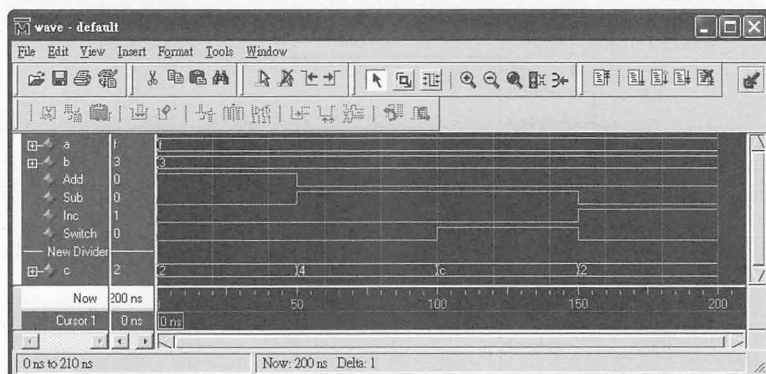
例 1 Func_AS.V，使用函數來設計相加、加 1 及相減處理的邏輯符號



```
// Func_AS.V: 使用函數來設計相加、加 1 及相減的處理
module Func_AS (a, b, Add, Sub, Inc, Switch, c);
input [3:0] a, b;
input Add, Sub, Inc, Switch;
output [3:0] c;
reg [3:0] c;

always @(a or b or Add or Sub or Inc or Switch)
begin
    if(Add) // 相加
        c = a + b;
    else
        if(Switch) // 交換 a, b
            c = Sub_Inc_Dec(b, a, Sub, Inc);
        else
            c = Sub_Inc_Dec(a, b, Sub, Inc);
    end

function [3:0] Sub_Inc_Dec;
input [3:0] a, b;
input Sub, Inc;
begin
    if(Sub) // 相減
        Sub_Inc_Dec = (a - b);
    else
        if(Inc) // 相加
            Sub_Inc_Dec = (a + b);
        else
            Sub_Inc_Dec = (a - 1'b1);
    end
endfunction
endmodule
```

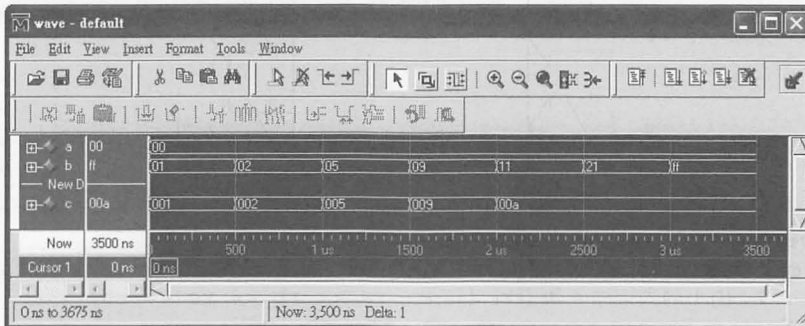


▲ Func_AS.V 的模擬結果

例 2 Add_LT10.V，使用函數設計最多只加 10 的處理的邏輯符號

```
// Add_LT10.V：使用函數設計最多只加 10 的處理
module Add_LT10 (a, b, c);
input  [7:0] a, b;
output [7:0] c;
wire    [7:0] c;

assign c = Add_LT_10 (a, b);
function [7:0] Add_LT_10;
input  [7:0] a, b;
reg    [7:0] temp;
begin
    if (b < 10)
        temp = b;
    else
        temp = 10; // 因為 temp 最大只會放 10，4 位元就夠用了
    Add_LT_10 = a + temp[3:0]; // temp[3:0] 可存數值範圍：0 ~ 15.
    // 因為 temp 必小於等於 10，所以只要使用 temp 的 [3:0] 就可以了。
end
endfunction
endmodule
```



▲ Add_LT10.V 的模擬結果



5.12 task 敘述

task 敘述與 function 敘述大致相同，都是將模組中重複用到的程式碼設計成函數或是副程式的形式，再去引用，而不用繁瑣的鍵入同樣的程式碼。

function 敘述至少要有一個或以上的 input 宣告，而 task 敘述可以擁有零個或是更多的 input、output 或是 inout 宣告。

- task 敘述是以關鍵字 task 與 endtask 宣告。
- task 敘述如同在 Quick Basic 程式語言或 Fortran 程式語言裡的 Sub-Routine 或 Pascal 程式語言的 Procedure。
- task 敘述同樣是被包含在 Verilog 階層式的架構中，同樣可以用階層式的名稱來引用到它。

範例 Sort4.V，由小到大排序 4 筆資料 (結果：ra <= rb <= rc <= rd) 的邏輯符號



```
// Copyright (c) 1995 by Synplicity, Inc.
// You may distribute freely, as long as this header remains attached.
// Sort4.V: 由小到大排序 4 筆資料 (結果: ra <= rb <= rc <= rd)
module Sort_4_Data(a, b, c, d, ra, rb, rc, rd);
parameter size = 4 - 1;
```

```

input  [size:0] a, b, c, d;
output [size:0] ra, rb, rc, rd;
reg    [size:0] ra, rb, rc, rd;

always @(a or b or c or d)
begin :Label_Name
// 少了:Label_Name 會發生: Expecting statement 的訊息
    reg [size:0] va, vb, vc, vd;

    {va,vb,vc,vd} = {a, b, c, d};

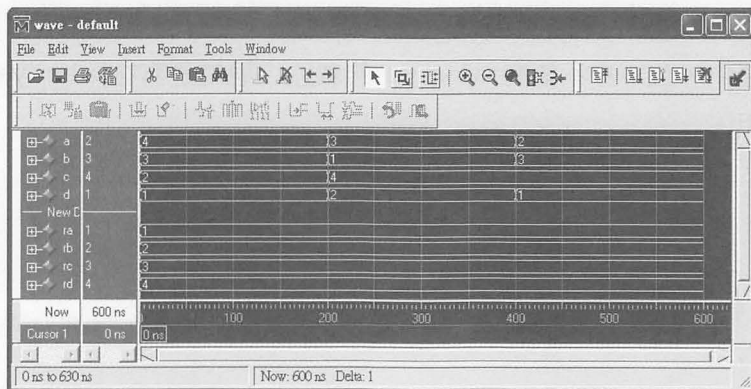
    Sort_2_Data (va, vc);
    Sort_2_Data (vb, vd);
    Sort_2_Data (va, vb);
    Sort_2_Data (vc, vd);
    Sort_2_Data (vb, vc);
    {ra, rb, rc, rd} = {va, vb, vc, vd};
end

task    Sort_2_Data;
inout [size:0] x, y;
reg    [size:0] temp;

    if (x > y)
    begin // 交換 (Swap)
        temp = x;
        x = y;
        y = temp;
    end
endtask

endmodule

```



▲ Sort4.V 的模擬結果



5.13 function 與 task 敘述的差異

(一) function 與 task 敘述主要不同的地方

函數 (function)	任務 (task)
function 可以引用其他的 function，但是不能引用其他的 task。	task 可以引用其他的 function 與 task。
function 至少要有一個或以上的 input 宣告。	task 可以擁有零個或是數個 input、output 或是 inout 宣告。
function 一定有傳回值。	task 沒有傳回值。

(二) function 與 task 敘述主要相同的地方

- task 敘述與 function 敘述大致相同，都是將模組中重複用到的程式碼設計成函數或是副程式的形式，再去引用，而不用繁瑣的鍵入同樣的程式碼。
- function 與 task 敘述都可以擁有本身的區域變數與暫存器，但不能擁有 wire 型態的變數。
- function 與 task 敘述只能用於行為模型 (Behavioral Model) 的敘述中，且本身不能有 always 的敘述，通常是寫在 always 區塊中被引用。

本章練習**Question 1 :**

為什麼有 if...else...敘述還要有 case 敘述、casex 敘述以及 casez 敘述？

Question 2 :

請用 function 敘述分別設計溫度轉換的電路。

華氏溫度 $F = \text{攝氏溫度 } C * 9 / 5 + 32$

攝氏溫度 $C = (\text{華氏溫度 } F - 32) * 5 / 9$

Question 3 :

reg [7:0] RegFile[3:0]; // 4 個 Register, 每個 Register 是
8 個 Bits

請將 RegFile[0] 給 5, RegFile[1] 給 2,

RegFile[2] 給 8 以及 RegFile[3] 給 6

再用 task 敘述設計具有排序 RegFile 這個 reg 陣列 (Arrays) 的電路。

PS Google 可以幫您找到解答。

